

Lastne vrednosti simetrične matrike s QR iteracijo z enojnim premikom

Urban Vesel

10. avgust 2026

1 Opis problema

Za simetrično matriko $A \in \mathbb{R}^{n \times n}$ iščemo vse njene lastne vrednosti in po želji pripadajoče lastne vektorje. Po spektralnem izreku ima vsaka taka matrika realne lastne vrednosti in ortonormirano bazo lastnih vektorjev, torej razcep $A = V\Lambda V^T$ z ortogonalno V in diagonalno Λ .

Lastnih vrednosti za $n > 4$ ne moremo poiskati z zaključeno formulo, zato jih računamo iterativno. Uporabimo QR iteracijo s premikom: matriko postopoma prevajamo v diagonalno obliko z zaporedjem ortogonalnih podobnostnih transformacij, ki spekter ohranjajo, obdiagonalne elemente pa potiskajo proti nič.

2 Opis rešitve

2.1 Osnovna ideja

Postopek ima dva koraka. Najprej matriko s Householderjevimi zrcaljenji prevedemo na simetrično *tridiagonalno* obliko T . To je končen postopek, ki matriko poenostavi, spektra pa ne spremeni. Nato tridiagonalno matriko iterativno diagonaliziramo: v vsakem koraku izberemo premik μ , izračunamo QR razcep premaknjene matrike in faktorja zmnožimo v obratnem vrstnem redu,

$$T - \mu I = QR, \quad T \leftarrow RQ + \mu I. \quad (1)$$

Ker je $RQ + \mu I = Q^T T Q$, je vsak korak ortogonalna podobnostna transformacija in lastne vrednosti se ohranijo natančno; premik vpliva samo na to, kako hitro se posamezen obdiagonalni element približa ničli. Ko zadnji obdiagonalni element postane zanemarljiv, se zadnja lastna vrednost odcepi (deflacija). Postopek nadaljujemo na manjši matriki.

Tridiagonalna oblika se pri koraku (1) ohrani. Matrika R ima nad diagonalo le dva neničelna pasova, Q je zgornja Hessenbergova, zato je tak tudi produkt RQ ; ker je hkrati simetričen, je nujno spet tridiagonalen. Zaradi tega lahko ta cel korak izvedemo neposredno na diagonalni in obdiagonalni, z zaporedjem Givensovih rotacij in brez hranjenja polne matrike.

2.2 Zakaj je enojni premik dobra izbira

Naloga predpisuje premik $\mu = t_{nn}$, torej zadnji diagonalni element aktivnega bloka. Iz $Q^T(T - \mu I) = R$ za zadnjo vrstico dobimo

$$q_n = r_{nn} (T - \mu I)^{-1} e_n, \quad (2)$$

kar pomeni, da je zadnji stolpec matrike Q natanko rezultat enega koraka inverzne iteracije s premikom μ . Ker je $\mu = e_n^T T e_n$ Rayleighov kvocient trenutnega približka, je celoten postopek

Rayleighova kvocientna iteracija, zapisana v bazi, ki jo sproti vrtimo. Pri simetričnih matrikah je Rayleighov kvocient od lastne vrednosti oddaljen kvadratično glede na kotno napako približka, zato je konvergenca *kubična*: če je napaka v enem koraku ε , je v naslednjem reda ε^3 . To se lepo vidi v zaporedju obdiagonalnega elementa med iteracijo,

$$6,5 \cdot 10^{-1} \rightarrow 8,6 \cdot 10^{-2} \rightarrow 3,8 \cdot 10^{-4} \rightarrow 3,8 \cdot 10^{-10} \rightarrow 7,7 \cdot 10^{-28},$$

kjer je vsak člen približno tretja potenca prejšnjega (slika 1).

Iz zadnjih treh členov zaporedja dobimo oceno reda konvergence $p = \frac{\log(e_{k+1}/e_k)}{\log(e_k/e_{k-1})} = 2,95$, kar se dobro ujema z teoretičnim pričakovanjem $p = 3$.

2.3 Deflacija in zanesljivost

Obdiagonalni element zanemarimo, ko velja $|e_i| \leq \varepsilon_{\text{stroj}}(|d_i| + |d_{i+1}|)$. Postavitev takega elementa na nič je motnja matrike z normo $|e_i|$, po Weylovem izreku se lastne vrednosti pri motnji premaknejo kvečjemu za njeno normo. Kriterij je torej povratno stabilen in vezan na velikost matrike, ne na absolutno vrednost.

Enojni premik pa ima slabost, da lahko obtiči. Za matriko $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ je $\mu = 0$ in korak matrike sploh ne spremeni, ker je Rayleighov kvocient točno na sredini med lastnima vrednostma. Zato iteracija spremlja padanje zadnjega obdiagonalnega elementa in ob zastoju preklopi na Wilkinsonov premik, ki konvergira vedno. Pri naših poskusih se je varovalo sprožilo v 3 od 1284 korakov (0,2 %), na naključnih matrikah torej skoraj nikoli. Izjema je antidiagonalna 2×2 matrika, pri kateri enojni premik dokazano obtiči: tam varovalo prevzame in iteracijo odblokira.

2.4 Zahtevnost

Redukcija na tridiagonalno obliko je najdražji del. Stane $\mathcal{O}(n^3)$ operacij. Posamezen korak iteracije pa se zaradi tridiagonalne strukture izvede v $\mathcal{O}(n)$ (oziroma $\mathcal{O}(n^2)$, če hkrati posodabljammo lastne vektorje), prostor pa je zgolj $\mathcal{O}(n)$, saj polne matrike nikoli ne hranimo. Ker zaradi kubične konvergence na lastno vrednost zadostuje le nekaj korakov (izmerili smo približno 2,6 do 3 korake), je iteracijski del skupaj $\mathcal{O}(n^2)$.

3 Rezultati

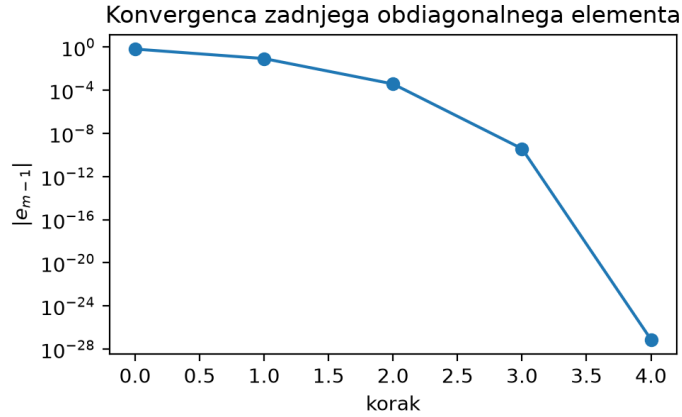
Natančnost smo preverili na naključnih simetričnih matrikah in jo primerjali z vgrajeno funkcijo `numpy.linalg.eigvalsh`:

n	$\max \Delta\lambda $	$\ V^T V - I\ _{\max}$	$\ AV - V\Lambda\ _{\max}$
10	$2,2 \cdot 10^{-15}$	$1,2 \cdot 10^{-15}$	$2,0 \cdot 10^{-15}$
40	$2,0 \cdot 10^{-14}$	$6,0 \cdot 10^{-15}$	$9,8 \cdot 10^{-15}$
100	$7,8 \cdot 10^{-14}$	$9,1 \cdot 10^{-15}$	$2,1 \cdot 10^{-14}$
200	$2,3 \cdot 10^{-13}$	$2,2 \cdot 10^{-14}$	$5,8 \cdot 10^{-14}$

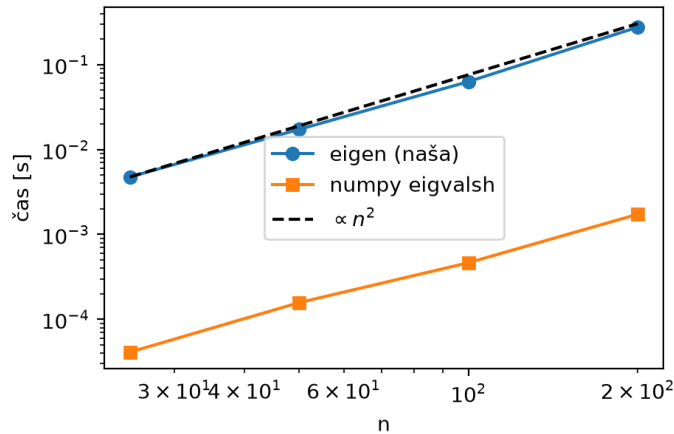
Tabela 1: Odstopanje lastnih vrednosti od oraklja, ortogonalnost izračunane baze in ostanek lastnega problema. Napaka narašča počasi z velikostjo matrike in ostaja blizu strojne natančnosti.

Hitrost smo primerjali z isto vgrajeno funkcijo (slika 2). Pri $n = 200$ je naša implementacija približno stokrat počasnejša, izmerjeni naklon pa je 1,84.

Meritve so bile opravljene z numpy 2.5.1; ker so razlike v zadnjih bitih odvisne od izvedbe BLAS, se lahko števila korakov in napake na drugi napravi razlikujejo za nekaj odstotkov.



Slika 1: Padanje zadnjega obdiagonalnega elementa po korakih iteracije za naključno simetrično matriko velikosti 12×12 . Navpična os je logaritemska; padec z 10^{-4} na 10^{-10} in nato na 10^{-28} v dveh korakih je značilen za kubično konvergenco.



Slika 2: Čas izračuna v odvisnosti od velikosti matrike. Izmerjeni naklon je približno 1,84, čeprav je metoda kot celota $\mathcal{O}(n^3)$: pri teh velikostih redukcija, izvedena z matričnimi operacijami knjižnice numpy, še ne prevlada, prevladuje pa iteracijski del, ki je v Pythonu izveden kot navadna zanka in je $\mathcal{O}(n^2)$. Vgrajena funkcija je zato približno stokrat hitrejša ob enaki asimptotiki.

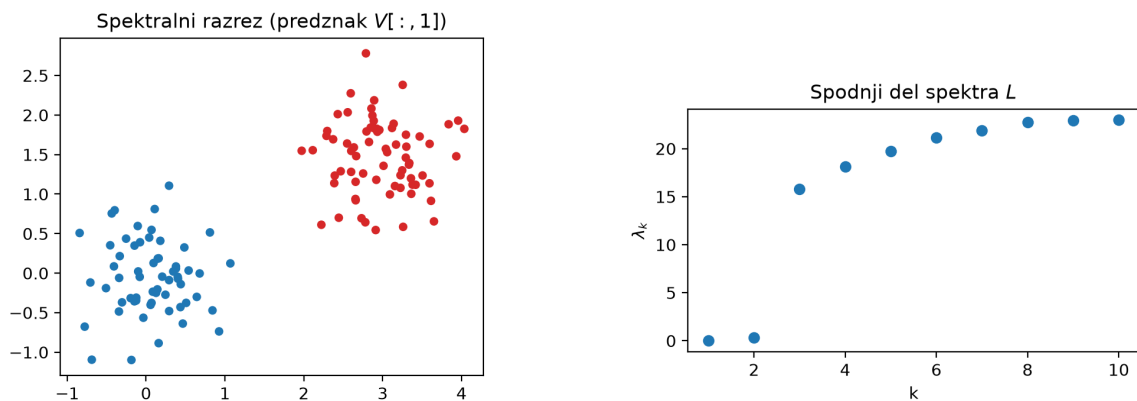
4 Primer uporabe: spektralno gručenje

Množico točk lahko razdelimo v gruče s pomočjo lastnih vektorjev. Točke povežemo v utežen graf, kjer je utež med dvema točkama $w_{ij} = \exp(-\|x_i - x_j\|^2/2\sigma^2)$, torej blizu ena za bližnji točki in skoraj nič za oddaljeni. Laplaceova matrika grafa je $L = D - W$, kjer je D diagonalna matrika vsot vrstic. Najmanjša lastna vrednost take matrike je vedno nič, pripadajoči lastni vektor pa je konstanten. Predznak komponent drugega lastnega vektorja (Fiedlerjevega) pa točke razdeli na dva dela tako, da je vsota uteži prerezanih povezav majhna.

Na dveh Gaussovih oblakih razrez pravilno razvrsti vse točke, med drugo in tretjo lastno vrednostjo pa je velika vrzel (vidno na sliki 3).

```
import numpy as np
from dnl_qr_premik import (EnojniPremik, eigen,
                           laplaceova_matrika, podobnostna_matrika)

L = laplaceova_matrika(podobnostna_matrika(X, sigma=0.8))
lastne, V = eigen(L, EnojniPremik(), vektorji=True)
gruca = V[:, 1] > 0      # razrez po Fiedlerjevem vektorju
```



Slika 3: Levo: 130 točk iz dveh Gaussovih oblakov, pobarvanih po predznaku Fiedlerjevega vektorja; razrez pravilno razvrsti vseh 130 točk. Desno: spodnji del spektra Laplaceove matrike. Prva lastna vrednost je $8,4 \cdot 10^{-15}$, torej numerično nič, med drugo in tretjo pa je velika vrzel ($\lambda_3/\lambda_2 \approx 45$), kar je znak, da sta gruči res dve.

5 Zaključek

Implementirali smo iskanje lastnih vrednosti in lastnih vektorjev simetrične matrike s QR iteracijo z enojnim premikom, pri čemer je posamezen korak izveden z Givensovimi rotacijami neposredno na diagonali in obdiagonali. Rezultati se z vgrajeno funkcijo ujema do reda 10^{-13} , izračunana baza pa ostane ortogonalna do strojne natančnosti. Kubična konvergenca se v poskusih jasno pokaže: na lastno vrednost v povprečju zadostujeta dva do trije koraki.

Glavna omejitev je hitrost. Ker je iteracijska zanka napisana v Pythonu, je implementacija pri $n = 200$ približno stokrat počasnejša od vgrajene funkcije, čeprav je asimptotika enaka. Naravna izboljšava bi bila prevod te zanke v prevedeno kodo ali uporaba implicitnega koraka, ki premaknjene matrike sploh ne izračuna eksplicitno.

Literatura

- [1] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4. izdaja, Johns Hopkins University Press, 2013, razdelek 8.3.
- [2] B. N. Parlett, *The Symmetric Eigenvalue Problem*, SIAM, 1998.
- [3] M. Vuk, *Numerična matematika v programskem jeziku Julia*, 2025, poglavje 8 in naloga 18.1.9.